

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Arquitectura de Computadores y Ensambladores 1



DOCUMENTACION TÉCNICA

Proyecto 1

Invernadero Inteligente IoT | Grupo 2

Emily Maritza Tepeu Guacamaya	202402955
Jackeline Stephany Rivera Argueta	202401685
Alison Melysa Pérez Blanco	202400023
José Fernando Ramírez Ambrocio	202400195
Diana Myriam Priscila Santizo Cáceres	202300484

TABLA DE CONTENIDO

INTRODUCCIÓN	5
REQUISITOS DEL SISTEMA	6
Requisitos de Hardware	6
Raspberry Pi	6
Memoria MicroSD	6
Fuente de poder	6
Requisitos de Software	6
Raspberry Pi OS	6
Python	6
Flask	6
Paho-mqtt	6
pymongo	6
RPI.GPIO	6
aarch64-linux-gnu-gcc	7
qemu-user	7
Dependencias Python	7
Estructura del proyecto	8
Subsistemas del invernadero	9
Monitoreo Ambiental	9
Monitoreo de Suelo por Área	10
Sistema de Riego	10
Sistema de Iluminación	11
Sistema de Ventilación	11
Sistema de Alarma	12
Centro de Control Físico	13
Conexiones Físicas	14
¿Qué es GPIO?	14
Numeración de pines	14
Inicialización de GPIO	15
Conexiones principales	15
Sensores (Entradas):	15
Actuadores (Salidas):	15
Centro de control:	16

¿Para qué es el Arduino?	16
¿Cómo se conecta?	16
¿Qué sensores van al Arduino?	16
Comunicación MQTT	17
¿Qué es MQTT?	17
Configuración del broker	17
Topics utilizados	17
Publicación (Raspberry Pi → Broker → Backend)	17
Suscripción (Raspberry Pi ← Broker ← Backend)	18
Publicación de lecturas	18
Recepción de comandos	18
Conexión de reconexión automática	19
Base de datos MongoDB	20
MongoDB Atlas	20
Dashboard Web	23
Tecnologías utilizadas	23
Endpoints de la API	23
Módulos ARM64	26
¿Por qué usar ensamblador?	26
Módulos	26
Biblioteca común (utils.s)	26
Formato de entrada (lecturas.csv)	27
Formato de salida	27
Compilación y ejecución	27
Raspberry Pi (ARM64 nativo)	27
PC (con QUEMU para pruebas)	27
Flujo de datos	28
Flujo principal (sensores → dashboard)	28
Flujo de comandos (dashboard → actuadores)	29
Flujo ARM64 (procesamiento estadístico)	29
Manual de Instalación	30
Requisitos previos	30
Hardware	30

Software en Raspberry Pi	30
Software en PC (para pruebas)	30
Clonar repositorio	30
Configuración de MongoDB Atlas	30
Configuración del Arduino	31
Ejecutar en la Raspberry Pi	31
Acceder al dashboard	31
Compilar y ejecutar módulos ARM64	31
Maqueta	32
Vista general	32
Centro de control	32
Interior del circuito	33
Evidencia de ejecución y pruebas	34
Consideraciones técnicas	36
Seguridad eléctrica	36
Rendimiento	36
Limitaciones conocidas	36
Preguntas frecuentes	37
¿Qué pasa si no hay conexión a internet?	37
¿Para qué sirve el Arduino?	37
¿Cuánto tiempo tarda en actualizarse el dashboard?	37
¿Dónde se guardan los resultados ARM64?	37

INTRODUCCIÓN

Este proyecto consiste en la implementación de un Invernadero Inteligente IoT utilizando una Raspberry Pi como unidad central de control. El sistema integra sensores ambientales, sensores de humedad de suelo, actuadores físicos, comunicación MQTT, persistencia en MongoDB Atlas y un dashboard web para monitoreo y control remoto.

El invernadero está dividido en dos áreas de cultivo independientes y un centro de control físico. Cada área cuenta con sensores y actuadores propios, permitiendo un control granular de las condiciones de crecimiento.

Objetivos del proyecto:

- Monitorear en tiempo real la temperatura, humedad ambiental, humedad de suelo, nivel de luz y nivel de gas.
- Controlar automáticamente el riego, ventilación, iluminación y alarmas.
- Permitir control manual desde panel físico (LCD + botones) y desde un dashboard web.
- Procesar datos estadísticamente mediante módulos en ensamblador ARM64.

REQUISITOS DEL SISTEMA

Requisitos de Hardware

Antes de ejecutar el programa, asegúrese de cumplir con los siguientes requisitos:

Raspberry Pi

- 3B, 3B+, 4B o 5 (ARM64)

Memoria MicroSD

- Mínimo: 16GB
- Recomendado: 32GB

Fuente de poder

- Mínimo: 5V / 3ª

Requisitos de Software

Raspberry Pi OS

- 64 bits (Bookworm o superior)

Python

- Versión 3.9+

Flask

- Versión 2.3+

Paho-mqtt

- Versión 1.6+

pymongo

- Versión 4.5+

RPI.GPIO

- Versión 0.7+

aarch64-linux-gnu-gcc

- Versión 13.0+ (para compilación cruzada)

qemu-user

- Versión 8.0+ (para pruebas en PC)

Dependencias Python

El proyecto utiliza la siguiente dependencia para que funcione de manera correcta, por lo que debe instalar lo siguiente:

```
bash
```

```
pip install flask paho-mqtt pymongo python-dotenv RPi.GPIO adafruit-circuitpython-dht
```

ESTRUCTURA DEL PROYECTO

El proyecto tiene diferentes carpetas y archivos los cuales son:

```

ARQUI1_GRUPO-2_P1/
├── README.md
├── ARM64/
│   ├── utils.s
│   ├── lecturas.csv
│   ├── modulo_1_media.s
│   ├── modulo_2_varianza.s
│   ├── modulo_3_anomalias.s
│   ├── modulo_4_prediccion.s
│   └── modulo_5_tendencia.s
├── backend/
│   ├── app.py
│   ├── arm64_runner.py
│   ├── config.py
│   ├── csv_manager.py
│   ├── database.py
│   ├── mqtt_client.py
│   ├── state.py
│   └── requirements.txt
├── dashboard/
│   ├── static/
│   │   ├── css/style.css
│   │   └── js/dashboard.js
│   └── templates/index.html
├── rasp/
│   ├── actuadores.py
│   ├── botones.py
│   ├── config_rasp.py
│   ├── lcd.py
│   ├── mqtt_rasp.py
│   ├── rasp_main.py
│   ├── sensores.py
│   ├── requirements_rasp.txt
│   └── Arduino/Esclavo.ino
└── documentacion/
    ├── ARM64/
    │   ├── Diagramas y documentación
    │   └── Documentación individual + GDB
    └── Diagramas y documentación

```

SUBSISTEMAS DEL INVERNADERO

El invernadero se divide principalmente en 3 zonas físicas: área de cultivo 1, área de cultivo 2 y el centro de control. Cada uno cuenta con componentes específicos que trabajan juntos para mantener las condiciones óptimas para las plantas.

Monitoreo Ambiental

Aquí se mide la temperatura, humedad de aire, nivel de luz y presencia de gases en el invernadero de forma constante.

- El sensor DHT11 mide temperatura y humedad ambiental, se lee cada 2 segundos y los valores se comparan con umbrales predefinidos.
- El LDR mide la cantidad de luz ambiental. Si hay poca luz, el sistema activa la iluminación de forma automática.
- El sensor MQ-135 detecta gases nocivos con humo, si supera un umbral crítico se activa la alarma y la ventilación de emergencia.

Variable	Sensor	Rango normal	Si no se cumple
Temperatura	DHT11	18-28°C	Activa ventilación
Humedad ambiente	DHT11	40-70%	Genera advertencia
Luz	LDR	>200 lux	Enciende luces si baja
Gas	MQ-135	<150 ppm	Emergencia si sube

```
# En rasp/sensores.py
import Adafruit_DHT

def leer_temperatura_humedad():
    humidity, temperature = Adafruit_DHT.read_retry(Adafruit_DHT.DHT11, cfg.PIN_DHT11)
    if humidity is not None and temperature is not None:
        return round(temperature, 1), int(humidity)
    return None, None
```

Monitoreo de Suelo por Área

Cada área de cultivo tiene su propio sensor de humedad de suelo, por lo que se puede regar solamente el área que lo necesita y así ahorrar agua ya que regar en exceso puede pudrir las raíces, si se riega menos puede secar la palta, entonces es mejor tener los sensores independientes así cada área recibe la cantidad exacta que necesita.

- El sensor mide la resistencia eléctrica del suelo. El suelo seco conduce menos electricidad, mientras que el suelo húmedo conduce más.
- El sistema clasifica el suelo en tres estados:

Estado	Lectura	Acción automática
SECO	>800 (seco)	Activar riego
NORMAL	400-800	No hacer nada
SATURADO	<400	Bloquear riego

Los sensores analógicos se conectan al Arduino. La Raspberry Pi lee los datos por el puerto serie USB. El Arduino envía una línea con el formato *GAS:512,SUELO1:680,SUELO2:450,LUZ:320*

```
def leer_sensores_arduino():
    if ser.in_waiting:
        linea = ser.readline().decode('utf-8').strip()
        partes = linea.split(',')
        for parte in partes:
            clave, valor = parte.split(':')
            valores[clave] = int(valor)
    return valores
```

Sistema de Riego

Se activan bombas de agua para regar las plantas cuando el suelo está seco.

- Modo automático: si el suelo está seco y pasó suficiente tiempo desde el último riego, se activa la bomba por 5 segundos.
- Modo manual: el usuario puede regar presionando un botón físico o desde el dashboard web.

```
def aplicar_control(suelo1, suelo2):
    ahora = time.time()
    pausa_ok = (ahora - _tiempo_ultimo_riego) > cfg.PAUSA_ENTRE_RIEGO

    if (suelo1 == "SECO" or suelo2 == "SECO") and pausa_ok and not _riego_activo:
        _activar_riego()
```

Sistema de Iluminación

Encienden luces LED cuando el invernadero está oscuro, simulando luz solar.

- El LDR mide la luz ambiental constantemente.
- Modo automático: si la luz baja de 200 lux las luces se encienden y cuando vuelve a subir se apagan.
- Modo manual: el usuario decide cuándo encender o apagar.

```
# En rasp/rasp_main.py - Lógica de iluminación
if estado["modo"] == "AUTOMATICO":
    if luz == "BAJO":
        estado["luces"] = True
        actuadores.luces(True)
    else:
        estado["luces"] = False
        actuadores.luces(False)
```

```
# En rasp/actuadores.py
def luces(encender):
    _estado["luces"] = encender
    _set(cfg.PIN_LED_LUZ_1, encender)
    _set(cfg.PIN_LED_LUZ_2, encender)
```

```
# En rasp/rasp_main.py - Botón físico
def btn_luces():
    estado["luces"] = not estado["luces"]
    actuadores.luces(estado["luces"])
    mqtt.publicar_comando_manual("LUCES", "ON" if estado["luces"] else "OFF")
```

Sistema de Ventilación

Se activa un ventilador para refrescar el invernadero o evacuar gases peligrosos.

- Se activa automáticamente cuando la temperatura supera los 30°C.
- También se activa si el sensor de gas detecta niveles peligrosos (emergencia).
- En emergencia por gas, el ventilador se enciende incluso si se está en modo manual.

```
# En rasp/rasp_main.py
if gas_estado == "GAS_EMERGENCIA":
    actuadores.ventilador(True)
elif estado["modo"] == "AUTOMATICO":
    if gas_estado == "GAS_ADVERTENCIA" or temp > cfg.UMBRAL_TEMP_ALTA:
        actuadores.ventilador(True)
    else:
        actuadores.ventilador(False)
```

Sistema de Alarma

Una alarma avisa si hay algún problema como de fuego o fuga de gas.

- Un buzzer emite un sonido cuando se detecta gas peligroso.

```
# En rasp/rasp_main.py - Activación por gas
if gas_estado == "GAS_EMERGENCIA":
    estado["alarma"] = True
    actuadores.buzzer(True)
    actuadores.set_led_estado("EMERGENCIA")
```

- Un LED rojo se enciende mostrando una emergencia.

```
# En rasp/actuadores.py
def set_led_estado(estado_global):
    _set(cfg.PIN_LED_VERDE, False)
    _set(cfg.PIN_LED_AMARILLO, False)
    _set(cfg.PIN_LED_ROJO, False)

    if estado_global == "NORMAL":
        _set(cfg.PIN_LED_VERDE, True)
    elif estado_global in ("ADVERTENCIA", "RIEGO_ACTIVO", "MODULO_MANUAL"):
        _set(cfg.PIN_LED_AMARILLO, True)
    elif estado_global == "EMERGENCIA":
        _set(cfg.PIN_LED_ROJO, True)
```

- La alarma se puede silenciar desde el dashboard o presionando el botón físico.

```
# En rasp/rasp_main.py - Silenciar
def btn_reset():
    estado["alarma"] = False
    actuadores.buzzer(False)
```

Centro de Control Físico

Si existe alguna falla de internet o del dashboard, el centro de control seguirá funcionando y permitirá operar el invernadero de forma local.

Botón	Función
Botón 1	Cambiar entre modo automático y manual
Botón 2	Activar riego manual (solo si está en manual)
Botón 3	Encender/apagar luces
Botón 4	Silenciar alarma

CONEXIONES FÍSICAS

¿Qué es GPIO?

GPIO significa "General Purpose Input/Output" (Entrada/Salida de Propósito General). Son pines en la Raspberry Pi que pueden configurarse como:

- Entrada: para leer sensores o botones
- Salida: para controlar actuadores como luces, ventilador o bomba.

Numeración de pines

La Raspberry Pi usa dos sistemas de numeración:

- BCM: número del GPIO interno
- BOARD: número físico del pin en el conector

Los pines GPIO se definen en *config_rasp.py* usando la numeración BCM, esta nomenclatura es la recomendada porque es independiente del modelo de la Raspberry Pi.

```
# En rasp/config_rasp.py
# Sensores digitales
PIN_DHT11      = 4      # DHT11 – temperatura y humedad
PIN_SUELO_1    = 16     # Sensor humedad suelo área 1
PIN_SUELO_2    = 26     # Sensor humedad suelo área 2
PIN_LDR        = 12     # Módulo LDR – luz

# Pines SPI para MCP3008 (sensor de gas analógico)
PIN_SPI_CLK    = 11     # CLK
PIN_SPI_MISO   = 9      # MISO (DOOUT del MCP3008)
PIN_SPI_MOSI   = 10     # MOSI (DIN del MCP3008)
PIN_SPI_CS     = 8      # CS/SS
CANAL_GAS      = 0      # MQ-135 conectado al canal 0

# Actuadores
PIN_RELE_BOMBA = 17     # Relé 1 → bomba de agua
PIN_RELE_VENT  = 22     # Relé 2 → ventilador
PIN_LED_LUZ_1  = 23     # LED iluminación área 1
PIN_LED_LUZ_2  = 24     # LED iluminación área 2
PIN_BUZZER     = 27     # Buzzer activo

# LEDs de estado
PIN_LED_VERDE  = 5      # Estado NORMAL
PIN_LED_AMARILLO = 6    # Estado ADVERTENCIA / RIEGO_ACTIVADO / MODO_MANUAL
PIN_LED_ROJO   = 13     # Estado EMERGENCIA

# Botones físicos (con pull-up interno)
PIN_BTN_MODAL = 14      # Botón 1 – cambiar AUTO/MANUAL
```

```

PIN_BTN_RIEGO    = 15    # Botón 2 – riego manual
PIN_BTN_LUCES    = 18    # Botón 3 – luces manual
PIN_BTN_RESET    = 20    # Botón 4 – silenciar alarma / reset

# LCD I2C
LCD_ADDRESS      = 0x27  # Dirección I2C
LCD_COLS         = 16
LCD_ROWS         = 2

```

Inicialización de GPIO

Al iniciar el programa, todos los pines de salida se configuran y se ponen el LOW apagados para evitar activaciones accidentales.

```

# En rasp/actuadores.py
def inicializar():
    salidas = [
        cfg.PIN_RELE_BOMBA, cfg.PIN_RELE_VENT,
        cfg.PIN_LED_LUZ_1,  cfg.PIN_LED_LUZ_2,
        cfg.PIN_BUZZER,
        cfg.PIN_LED_VERDE,  cfg.PIN_LED_AMARILLO, cfg.PIN_LED_ROJO,
    ]
    for pin in salidas:
        GPIO.setup(pin, GPIO.OUT)
        GPIO.output(pin, GPIO.LOW)

```

Conexiones principales

Sensores (Entradas):

- DHT11 (temperatura/humedad) → GPIO 4. Este sensor usa un protocolo un cable y necesita una resistencia pull-up de 4.7k Ω .
- Sensores de humedad de suelo (Área 1 y 2) → GPIO 17 y 27. Miden resistencia del suelo.
- LDR (luz) → GPIO 22. Un divisor de voltaje con una resistencia convierte la luz en voltaje.

Actuadores (Salidas):

- Bombas de agua → GPIO 18 y 19 a través de relés. Los relés aíslan la Raspberry Pi de la fuente externa de 5V que alimenta las bombas.
- Ventilador → GPIO 20 a través de un relé.
- LEDs de iluminación → GPIO 21 y 26. Se encienden directamente con resistencia de 220 Ω .

- LEDs de estado (verde, amarillo, rojo) → GPIO 12, 13, 24.
- Buzzer → GPIO 25.

Centro de control:

- LCD 16x2 → I2C (GPIO 2 y 3). Usa solo 2 pines para comunicación.
- Botones → GPIO 5, 6, 16, 7. Usan resistencias pull-up internas.

¿Para qué es el Arduino?

La Raspberry Pi no tiene entradas analógicas. Los sensores de suelo, LDR y gas dan valores analógicos, por esto se usa el Arduino ya que este si cuenta con entradas analógicas (A0 – A5).

¿Cómo se conecta?

- Arduino se conectar por USB a la Raspberry Pi
- La Raspberry Pi lee los datos por puerto serial
- El Arduino envía una línea de texto cada 2 segundos con formato:
GAS:512,SUELO1:680,SUELO2:450,LUZ:320.}

¿Qué sensores van al Arduino?

- MQ-135 (gas) → pin A0
- Sensor suelo área 1 → pin A1
- Sensor suelo área 2 → pin A2
- LDR (luz) → pin A3

COMUNICACIÓN MQTT

¿Qué es MQTT?

MQTT es un protocolo de comunicación ligero para IoT, funciona con un modelo publicador/suscriptor.

- Publicador: envía mensajes a un topic.
- Suscriptor: recibe un mensaje de un tema.
- Broker: es el servidor central que recibe y manda los mensajes.

La Raspberry Pi publica las lecturas de los sensores a topics como *invernadero/sensores/temperatura*. El backend está suscrito a estos topics y recibe los datos en tiempo real. Cuando el usuario presiona un botón en el dashboard, el backend publica un comando a un topic como *invernadero/actuadores/riego*. La Raspberry Pi está suscrita a ese topic y ejecuta la acción.

Configuración del broker

El sistema utiliza un bróker MQTT público y gratuito. Todos los topics comparten el prefijo *InvernaderoG2_ARQUI1/* para organizar la comunicación.

```
# En rasp/config_rasp.py
MQTT_BROKER = "broker.emqx.io"
MQTT_PORT   = 1883
MQTT_PREFIX = "InvernaderoG2_ARQUI1"
```

Topics utilizados

Publicación (Raspberry Pi → Broker → Backend)

- InvernaderoG2_ARQUI1/sensores/temperatura
- InvernaderoG2_ARQUI1/sensores/humedad_ambiente
- InvernaderoG2_ARQUI1/sensores/humedad_suelo_area1
- InvernaderoG2_ARQUI1/sensores/humedad_suelo_area2
- InvernaderoG2_ARQUI1/sensores/luz
- InvernaderoG2_ARQUI1/sensores/gas
- InvernaderoG2_ARQUI1/estado/global

Suscripción (Raspberry Pi ← Broker ← Backend)

- InvernaderoG2_ARQUI1/actuadores/riego_area1
- InvernaderoG2_ARQUI1/actuadores/riego_area2
- InvernaderoG2_ARQUI1/actuadores/ventilador
- InvernaderoG2_ARQUI1/actuadores/luces
- InvernaderoG2_ARQUI1/actuadores/alarma
- InvernaderoG2_ARQUI1/control/modo
- InvernaderoG2_ARQUI1/control/reset

Publicación de lecturas

Cada sensor tiene su propia función de publicación, los mensajes se envían en formato JSON con timestamp y metadatos.

```
# En rasp/mqtt_rasp.py
def publicar_temperatura(valor):
    _pub(cfg.TOPIC_TEMPERATURA, {
        "valor": valor,
        "unidad": "C",
        "timestamp": datetime.now().isoformat(),
        "origen": "DHT11"
    })

def publicar_humedad_suelo(area, valor, estado):
    topic = cfg.TOPIC_HUMEDAD_SUELO_1 if area == 1 else cfg.TOPIC_HUMEDAD_SUELO_2
    _pub(topic, {
        "valor": valor,
        "estado": estado,
        "area": area,
        "timestamp": datetime.now().isoformat(),
        "origen": f"sensor_suelo_area{area}"
    })
```

Recepción de comandos

La Raspberry Pi está suscrita a todos los topics con el prefijo del proyecto. Cuando llega un comando, se ejecuta la acción correspondiente.

```
# En rasp/mqtt_rasp.py
def on_message(client, userdata, msg):
    payload = json.loads(msg.payload.decode())
    topic = msg.topic
    if topic in (cfg.TOPIC_CONTROL_REMOTO, cfg.TOPIC_CONTROL_MANUAL):
        if _cb_comando:
            _cb_comando(payload)
```

```
# En rasp/rasp_main.py
def procesar_comando(payload):
    accion = payload.get("accion", "").upper()
    valor = payload.get("valor", "").upper()

    if accion == "VENTILADOR":
        enc = valor == "ON"
        estado["ventilador"] = enc
        actuadores.ventilador(enc)
    elif accion == "LUCES":
        enc = valor == "ON"
        estado["luces"] = enc
        actuadores.luces(enc)
    elif accion == "RIEGO_AREA1" and valor == "ON":
        _activar_riego()
```

Conexión de reconexión automática

El cliente MQTT se configura con reconexión automática en caso de pérdida de conexión.

```
# En rasp/mqtt_rasp.py
def iniciar(cb_comando=None):
    client_id = f"InvernaderoG2_rasp-{random.randint(0, 9999)}"
    client = mqtt_client.Client(client_id=client_id)
    client.on_connect = on_connect
    client.on_disconnect = on_disconnect
    client.on_message = on_message
    client.reconnect_delay_set(min_delay=1, max_delay=30)
    client.connect(cfg.MQTT_BROKER, cfg.MQTT_PORT, keepalive=60)
    client.loop_start()
```

BASE DE DATOS MONGODB

MongoDB Atlas

MongoDB es una base de datos NoSQL que guarda datos en formato JSON, a diferencia de las bases de datos SQL, esta no necesita tablas con esquemas fijos.

MongoDB Atlas es la versión en la nube de MongoDB la cual es gratuita para proyectos pequeños y permite acceder a los datos desde cualquier lugar sin preocuparse por respaldos o mantenimiento del servidor. Lo siguiente son las colecciones de datos que se guardan:

- *sensor_readings*: guarda cada lectura de sensores con su timestamp y permite generar gráficas históricas en el dashboard.

```
{
  "timestamp": ISODate("2026-06-10T14:30:00Z"),
  "temperatura": 24.5,
  "humedad_ambiente": 60,
  "humedad_suelo_1": 45,
  "humedad_suelo_2": 48,
  "nivel_luz": 820,
  "nivel_gas": 120
}
```

- *events*: registra eventos importantes como alertas, emergencias, activación de riego y cambios de estado.

```
{
  "timestamp": ISODate("2026-06-10T14:30:00Z"),
  "tipo": "ALERTA",
  "descripcion": "Temperatura superó umbral (32°C)",
  "origen": "SISTEMA"
}
```

- *commands*: guarda todos los comandos enviados desde el dashboard o con los botones físicos.

```
{
  "timestamp": ISODate("2026-06-10T14:32:00Z"),
  "comando": "RIEGO_AREA_1",
  "origen": "DASHBOARD",
  "estado": "EJECUTADO"
}
```

- *system_status*: guarda el estado actual del sistema.

```
{
  "timestamp": ISODate("2026-06-10T14:30:00Z"),
  "estado_global": "NORMAL",
  "modo": "AUTOMATICO",
  "riego_activo": false,
  "ventilacion_activa": false,
  "luces_encendidas": false,
  "alarma_activa": false
}
```

- *actuador_logs*: registra cada vez que se activa un actuador.

```
{
  "timestamp": ISODate("2026-06-10T14:25:00Z"),
  "actuador": "BOMBA_AREA_1",
  "accion": "ACTIVADO",
  "duracion_segundos": 5,
  "origen": "AUTOMATICO"
}
```

- *arm64_results*: guarda los resultados de los 5 módulos ARM64 y el dashboard consulta esta colección para mostrar los análisis estadísticos.

```
{
  "timestamp": ISODate("2026-06-10T14:35:00Z"),
  "modulo": "ANOMALY_DETECTION",
  "media": 111,
  "std_dev": 11,
  "anomalies": 0,
  "risk": "NORMAL"
}
```

A continuación se muestra como ese que llegan los datos a la base de datos de forma correcta.

ADD DATAUPDATEDELETEEXPORT CODE

251 - 4 of 4

★ events	_id ObjectId	tipo String	valor Int32	msg String	timestamp String	
1	ObjectId('6a205fa5e4ab30...')	"EMERGENCIA_GAS"	297	"¡Gas peligroso detectad...	"2026-06-03T11:08:53.396...	  
2	ObjectId('6a205fa8e4ab30...')	"RESOLUCION_EMERGENCIA"	No field	"Gas regresó a niveles s...	"2026-06-03T11:08:55.861...	  
3	ObjectId('6a205fbfe4ab30...')	"EMERGENCIA_GAS"	331	"¡Gas peligroso detectad...	"2026-06-03T11:09:19.477...	  
4	ObjectId('6a205fc2e4ab30...')	"RESOLUCION_EMERGENCIA"	No field	"Gas regresó a niveles s...	"2026-06-03T11:09:21.934...	  

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
actuator_logs	-	32.77 kB	398.00 B	2	199.00 B	1	24.58 kB
arm64_results	-	4.10 kB	0 B	0	0 B	1	4.10 kB
commands	-	32.77 kB	0 B	0	0 B	1	32.77 kB
events	-	24.58 kB	374.00 B	2	187.00 B	1	24.58 kB
sensor_readings	-	32.77 kB	816.00 B	2	408.00 B	1	24.58 kB
system_status	-	24.58 kB	227.00 B	1	227.00 B	1	36.86 kB

DASHBOARD WEB

El dashboard es una página web que permite:

- Monitorear el invernadero en tiempo real
- Controlar los actuadores
- Ver gráficas históricas de los sensores
- Consultar resultados de los módulos AR364

El archivo *dashboard.js* tiene un *setInterval* lo cual hace una petición cada 3 segundos hacia */api/datos* y actualiza los valores en la pantalla.

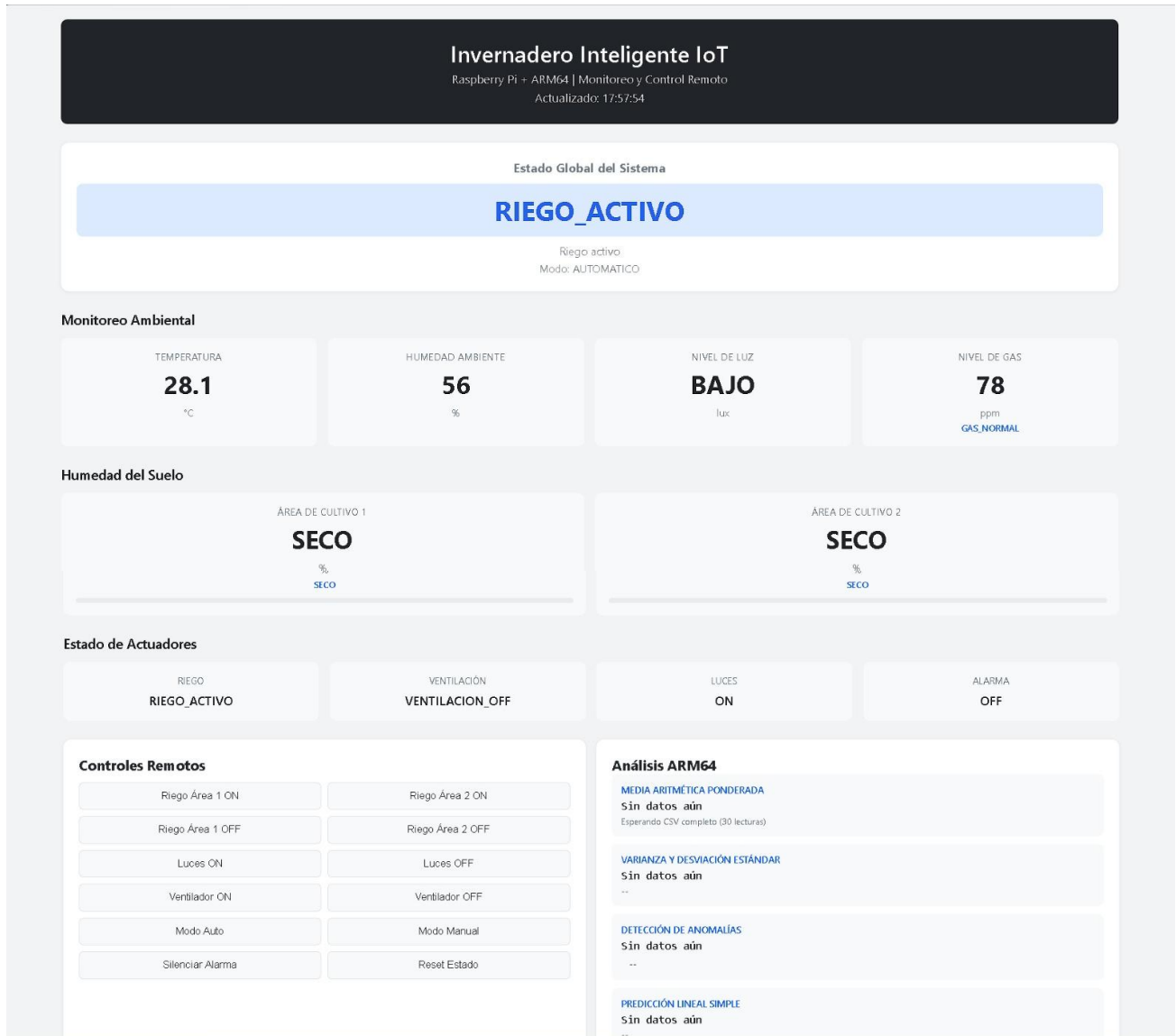
Tecnologías utilizadas

Tecnología	Para qué sirve
Flask	Backend en Python. Crea la API que el frontend consulta.
HTML5	Estructura de la página.
CSS3	Estilos visuales (colores, tamaños, posiciones).
JavaScript	Lógica del cliente: actualiza datos, maneja botones, dibuja gráficas.
Chart.js	Biblioteca para dibujar gráficas de líneas.
Fetch API	Permite al frontend pedir datos al backend sin recargar la página.

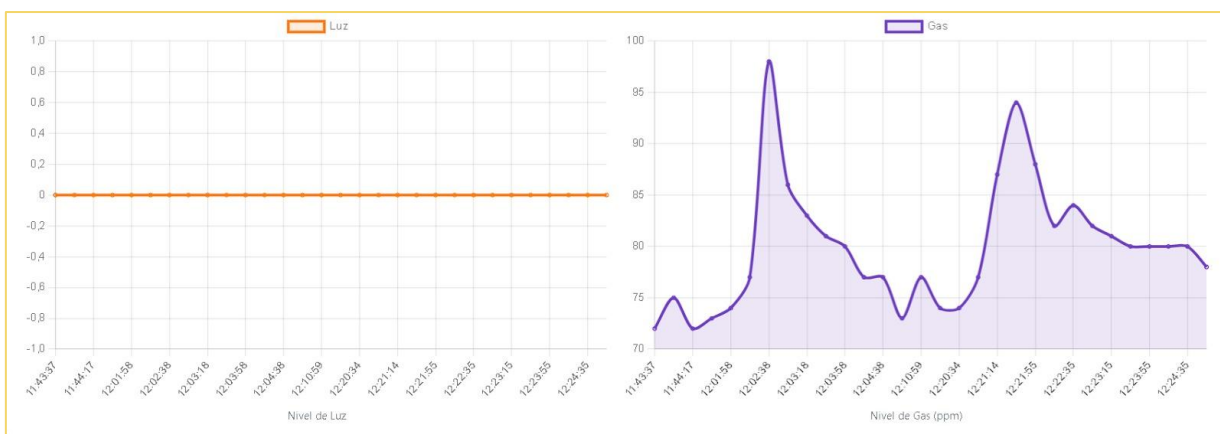
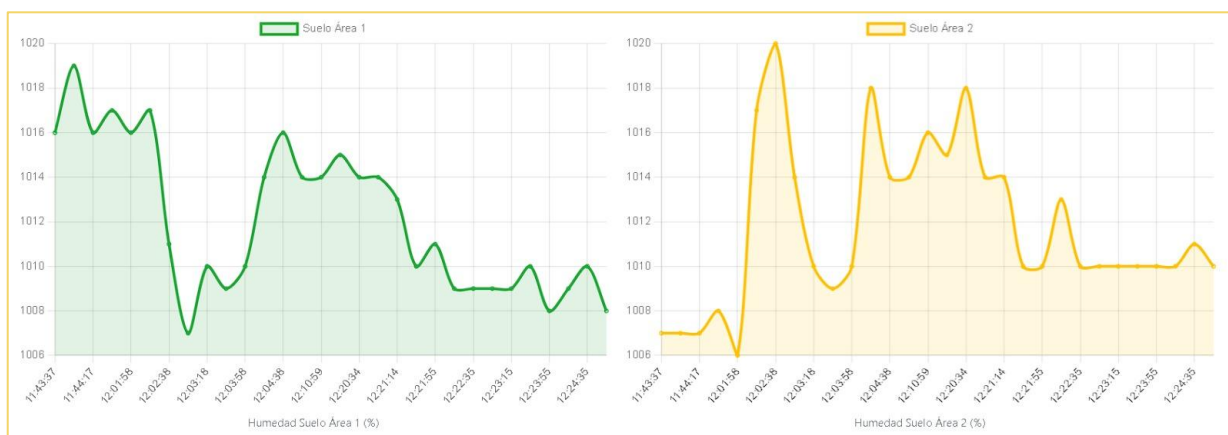
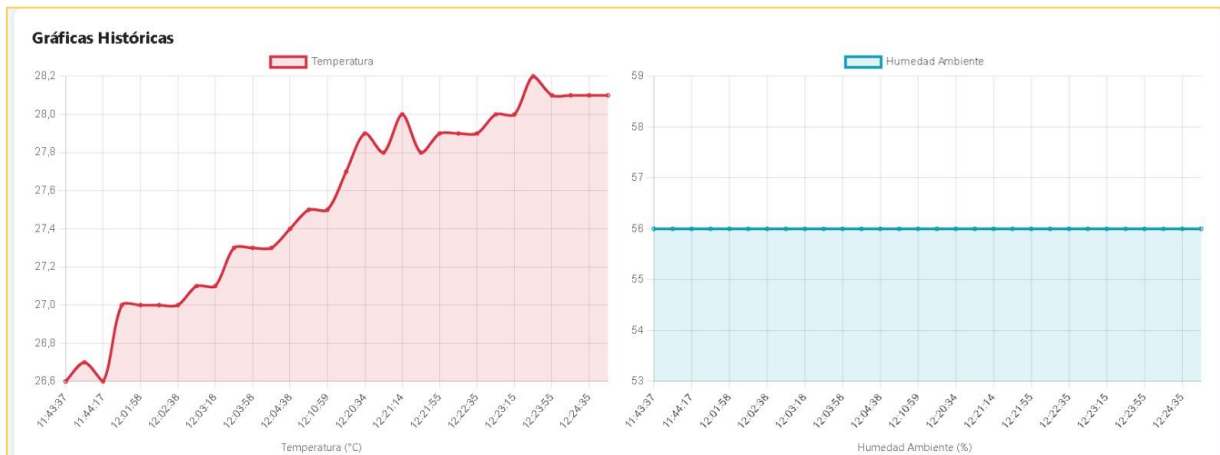
Endpoints de la API

Endpoint	Método	Qué devuelve
/	GET	La página HTML del dashboard
/api/datos	GET	Última lectura de sensores en JSON
/api/comando	POST	Recibe un comando y lo envía por MQTT
/api/historial	GET	Últimas 30 lecturas para las gráficas
/api/eventos	GET	Lista de eventos recientes
/api/arm64	GET	Resultados de los módulos ARM64

A continuación se muestran algunas imágenes del dashboard web funcionando de forma correcta, en donde se puede apreciar el estado del sistema con su monitoreo de cada sensor y sus datos.



Las siguientes imágenes muestran las gráficas históricas de cada sensor (temperatura, humedad ambiental, suelo área 1 y 2, luz y gas), mostrando los últimos 30 registros de cada uno.



MÓDULOS ARM64

Arm64 (también llamada aarch64) es la arquitectura de 64 bits de los procesadores ARM. La Raspberry usa ARM64.

¿Por qué usar ensamblador?

El proyecto requiere que cada integrante desarrolle un módulo en lenguaje ensamblador para demostrar comprensión de:

- Registros
- Memoria
- Ciclos y saltos condicionales
- Subrutinas
- Llamadas al sistema

Módulos

El proyecto cuenta con 5 módulos que procesan estadísticamente los 30 datos del archivo *lecturas.csv* que son los siguientes:

Módulo	Función	Fórmulas
Módulo 1	Media ponderada	$\Sigma(X_i * W_i) / \Sigma W_i$, con $W_i = 1..30$
Módulo 2	Varianza y desviación	$\sigma^2 = \Sigma(X - \mu)^2 / N$, $\sigma = \sqrt{\sigma^2}$
Módulo 3	Detección de anomalías	$Z = (X - \mu) / \sigma$, anomalía si $Z \geq 2$
Módulo 4	Predicción lineal	$\text{Próximo} = X_{\text{final}} + (X_{\text{final}} - X_{\text{inicial}}) / (N-1)$
Módulo 5	Tendencia acumulada	Cuenta incrementos, decrementos y rachas

Biblioteca común (utils.s)

Todos los módulos comparten *utils.s* que proporciona:

- *leer_datos*: lee el CSV y extra una columna
- *int_a_ascii*: convierte un número a texto
- *ascii_a_int*: convierte un texto a número
- *escribir_archivo*: guardo resultados en un *.txt*

Formato de entrada (lecturas.csv)

El archivo tiene 30 filas con datos reales de sensores:

```
ID,TEMP,HUM_AIRE,HUM_SUELO_1,HUM_SUELO_2,LUZ,GAS,RIEGO_1,RIEGO_2
1,24.6,57,1,1,0,74,0,0
2,24.4,57,1,1,0,73,0,0
3,24.5,57,1,1,0,74,0,0
4,24.4,57,1,1,0,74,0,0
5,24.4,57,1,1,0,74,0,0
6,24.4,57,1,1,0,74,0,0
7,24.5,57,1,1,0,72,0,0
8,24.5,57,1,1,0,75,0,0
```

Formato de salida

Cuando el programa termina de acuerdo a cada módulo, se genera el archivo de salida que muestra diferentes datos calculados, el siguiente ejemplo muestra los resultados del módulo 3:

```
emily-tepeu@emilytpu-pc:~/Es
MODULE=ANOMALY_DETECTION
TOTAL_VALUES=30
MEAN=111
STD_DEV=11
ANOMALIES=0
SYSTEM_RISK=NORMAL
```

Compilación y ejecución

Raspberry Pi (ARM64 nativo)

```
bash

as -o utils.o utils.s
as -o modulo_3_anomalias.o modulo_3_anomalias.s
ld -o modulo_3_anomalias modulo_3_anomalias.o utils.o
./modulo_3_anomalias
```

PC (con QEMU para pruebas)

```
bash

aarch64-linux-gnu-as -o utils.o utils.s
aarch64-linux-gnu-as -o modulo_3_anomalias.o modulo_3_anomalias.s
aarch64-linux-gnu-ld -o modulo_3_anomalias modulo_3_anomalias.o utils.o
qemu-aarch64 ./modulo_3_anomalias
```

FLUJO DE DATOS

El flujo de datos es el camino que sigue la información desde el sensor físico cuando mide algo hasta cuando se muestra en el dashboard o sea guarda en la base de datos.

Flujo principal (sensores → dashboard)

1. Lectura de sensores

- La Raspberry Pi lee el DHT11 directamente por GPIO.
- El Arduino lee los sensores analógicos (suelo, luz, gas) y envía los datos por USB serie.

2. Procesamiento en Raspberry Pi

- *rasp_main.py* recibe los datos del Arduino y del DHT11.
- Aplica la lógica de control (umbrales, estados, decisiones).

3. Publicación MQTT

- La Raspberry Pi publica las lecturas a topics MQTT como *invernadero/sensores/temperatura*.
- El bróker MQTT recibe y reenvía los mensajes.

4. Recepción en backend

- El backend (Flask) está suscrito a los topics MQTT.
- Recibe los datos y los guarda en MongoDB Atlas.

5. Consulta del dashboard

- El dashboard (JavaScript) consulta el endpoint `/api/datos` cada 3 segundos.
- El backend responde con la última lectura desde MongoDB.

6. Visualización

- El dashboard actualiza los valores en pantalla.
- Las gráficas históricas consultan `/api/historial` para obtener los últimos 30 registros.

Flujo de comandos (dashboard → actuadores)

1. El usuario presiona el botón
 - El usuario hace clic en “Riego área 1” en el dashboard.
2. El dashboard envía el comando
 - JavaScript hace una petición POST a */api/comando* con el comando.
3. Backend publica por MQTT
 - Flask recibe el comando y lo publica al topic *invernadero/actuadores/riego_area1*.
4. Raspberry Pi recibe
 - *mqtt_rasp.py* está suscrito al topic y recibe el mensaje.
 - Llama a la función *procesar_comando()* en *rasp_main.py*.
5. Ejecución del actuador
 - *rasp_main.py* llama a *actuadores.bomba(True)*.
 - La bomba se enciende por 5 segundos y luego se apaga.

Flujo ARM64 (procesamiento estadístico)

1. Generación del CSV
 - Python toma los últimos 30 datos reales de los sensores en *csv_manager.py*.
 - Los escribe en *lecturas.csv* con el formato requerido.
2. Ejecución de módulos
 - *arm64_runner.py* ejecuta los 5 módulos ARM64 usando *subprocess*.
 - Cada módulo lee *lecturas.csv*, procesa las columnas y genera *resultado_*.txt*.
3. Lectura de resultados
 - Python lee los archivos de resultados.
4. Almacenamiento en MongoDB
 - Los resultados se guardan en la colección *arm64_results*.
5. Visualización en el dashboard
 - El dashboard consulta a */api/arm64* y muestra los resultados en la sección “Análisis ARM64”.

MANUAL DE INSTALACIÓN

Requisitos previos

Hardware

- Raspberry Pi 4 con Raspberry Pi OS 64 bits instalado
- Arduino Uno (para sensores analógicos)

Software en Raspberry Pi

```
bash

# Actualizar sistema
sudo apt update && sudo apt upgrade -y

# Instalar Python y pip
sudo apt install python3 python3-pip git -y

# Instalar librerías GPIO y dependencias
sudo apt install python3-rpi.gpio -y
pip3 install paho-mqtt pymongo flask python-dotenv pyserial adafruit-circuitpython-dht
```

Software en PC (para pruebas)

```
bash

sudo apt install aarch64-linux-gnu-gcc aarch64-linux-gnu-binutils qemu-user gdb-multiarch
```

Clonar repositorio

```
bash

git clone https://github.com/EmilyTepeu00/ARQUI1_GRUPO-2_P1.git
cd ARQUI1_GRUPO-2_P1
```

Configuración de MongoDB Atlas

1. Crear cuenta en MongoDB Atlas
2. Crear cluster gratuito (M0)
3. Obtener URI de conexión
4. Crear archivo `.env` en la carpeta *backend/* de la siguiente manera:

```
bash
```

```
MONGODB_URI=mongodb+srv://usuario:password@cluster0.mongodb.net/  
MONGODB_DB=invernadero_db
```

Configuración del Arduino

1. Conectar el Arduino por USB a la Raspberry Pi
2. Cargar el código *Esclavo.ino* desde Arduino IDE o usando *Arduino-cli*
3. Verificar el puerto serie: `ls /dev/ttyUSB*` o `/dev/ttyAMC*`

Ejecutar en la Raspberry Pi

Terminal 1 (programa principal):

```
bash
```

```
cd ~/ARQUI1_GRUPO-2_P1/rasp  
python3 rasp_main.py
```

Terminal 2 (backend Flask):

```
bash
```

```
cd ~/ARQUI1_GRUPO-2_P1/backend  
python3 app.py
```

Acceder al dashboard

Abrir el navegador en [http://\[IP_DE_RASPBERRY\]:5000](http://[IP_DE_RASPBERRY]:5000) y ahí mismo podrá visualizar la página del dashboard web, en donde puede encontrar diferentes botones, paneles y gráficas.

Compilar y ejecutar módulos ARM64

```
bash
```

```
cd ~/ARQUI1_GRUPO-2_P1/ARM64  
as -o utils.o utils.s  
as -o modulo_3_anomalias.o modulo_3_anomalias.s  
ld -o modulo_3_anomalias modulo_3_anomalias.o utils.o  
./modulo_3_anomalias  
cat resultado_anomalias.txt
```

MAQUETA

Vista general

La maqueta cuenta con todas las especificaciones requeridas del invernadero y sus áreas, sensores, manguera de riego, leds de iluminación y centro de control.



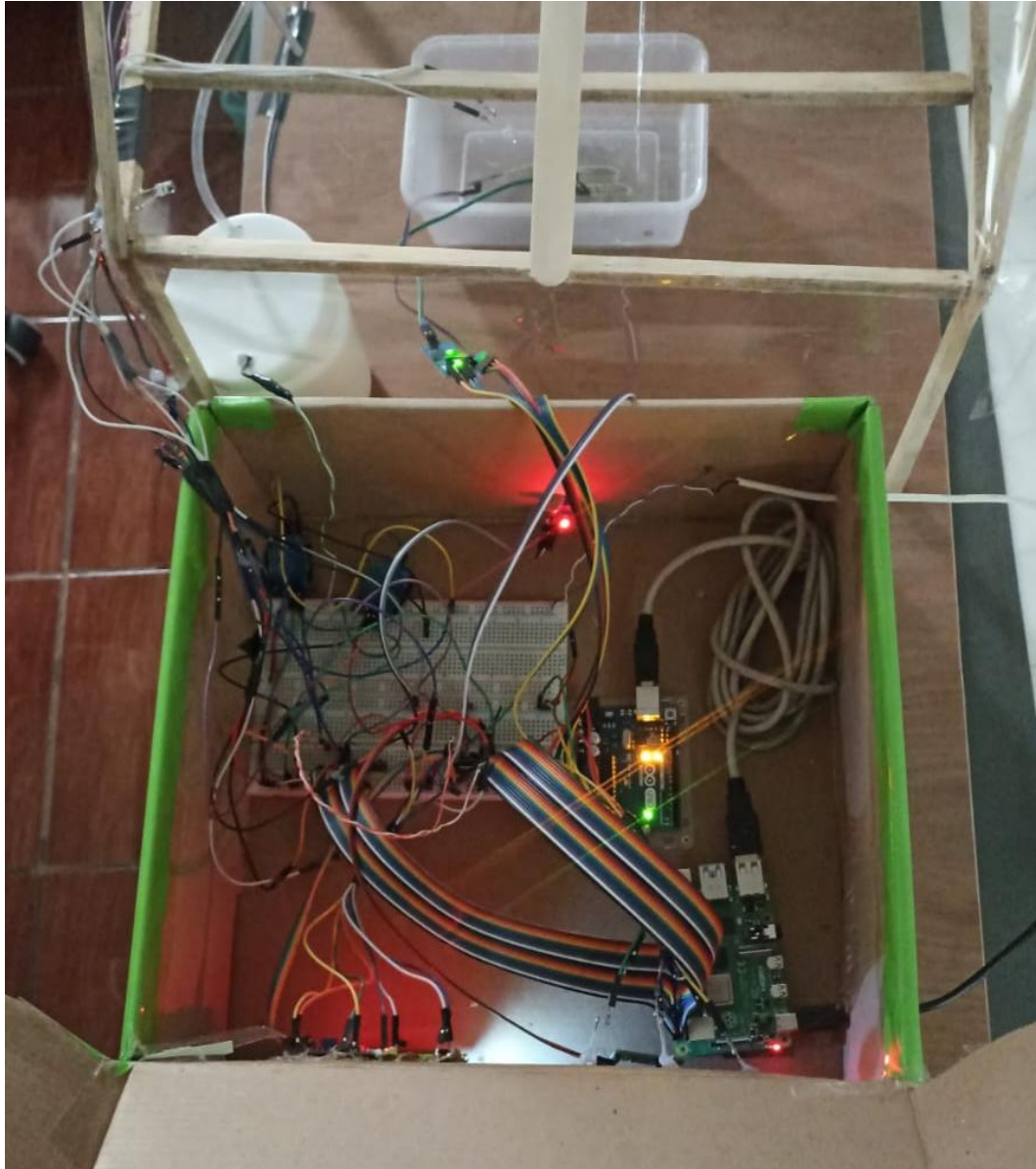
Centro de control

El panel físico incluye: LCD 16x2 para mostrar la temperatura, humedad, estado de suelo y gas, así mismo cuenta con 4 botones para modo auto/manual, riego manual, luces y silenciar alarma.



Interior del circuito

Dentro de la caja del centro de control, se encuentra la Raspberry Pi, protoboard, relés para bombas demás conexiones de cableado para que el sistema funcione de manera correcta.



EVIDENCIA DE EJECUCIÓN Y PRUEBAS

A continuación se muestran varias pruebas de ejecución con la Raspberry Pi para verificar su funcionamiento junto al backend.

Se muestra como la Raspberry Pi recibe datos de los sensores y los publica mediante MQTT y el backend responde consultas del dashboard. Se observa el estado general del sistema y las lecturas en tiempo real.

The screenshot shows a terminal window on a Raspberry Pi with two panes. The left pane displays a series of GET requests to a backend API, including endpoints for /api/estado, /api/historial, /api/eventos, /api/comandos, /api/csv, and /api/arm64. The right pane shows MQTT messages being published to various topics, including sensor data for temperature, humidity, and gas, as well as system status updates. The terminal output indicates that the system is currently in a 'NORMAL' state with 'RIEGO=OFF'.

En la siguiente imagen se muestra como el sistema detecta cambios en las condiciones del invernadero y activa el riego automático cuando el suelo está seco, así mismo cambia el estado de emergencia cuando el sensor de gas supera el umbral establecido.

This screenshot shows the terminal output after a state change. The left pane shows GET requests to the backend API, and the right pane shows MQTT messages. The system status has changed to 'EMERGENCIA' (Emergency) with 'RIEGO=OFF'. The terminal output indicates that the system has detected a change in conditions and has activated the automatic irrigation system.

CONSIDERACIONES TÉCNICAS

Seguridad eléctrica

- No aplicar 5V directamente a GPIO (usar resistencias o divisores).
- Usar fuente externa para bombas y ventilador (no alimentar desde GPIO).

Usar relés o transistores para controlar cargas de alta corriente.

Colocar resistencia pull-up de $4.7k\Omega$ en sensores DHT y MQ-135.

Rendimiento

- El dashboard actualiza cada 3 segundos.
- Los módulos ARM64 procesan 30 datos fijos.
- MongoDB Atlas requiere conexión a internet.

Limitaciones conocidas

- El sensor DHT11 tiene precisión limitada ($\pm 2^{\circ}\text{C}$, $\pm 5\%$ humedad).
- El Arduino esclavo es necesario porque la Raspberry Pi no tiene entradas analógicas.
- Los módulos ARM64 trabajan con enteros, no con decimales (se escala multiplicando por 10).

PREGUNTAS FRECUENTES

¿Qué pasa si no hay conexión a internet?

MongoDB no podrá guardar datos y el dashboard no podrá mostrar los datos históricos.

¿Para qué sirve el Arduino?

La Raspberry Pi no cuenta con entradas analógicas, por lo que el Arduino lee las señales analógicas de sensores (gas, suelo, luz) y las envía por USB serie.

¿Cuánto tiempo tarda en actualizarse el dashboard?

El tiempo es de 3 segundos, si se desea cambiar se debe modificar la línea `setInterval(actualizarDashboard, 3000)` en el archivo `dashboard.js`

¿Dónde se guardan los resultados ARM64?

En la base de datos de MongoDB Atlas, colección `arm64_results`. También se generan archivos `.txt` locales.

